Aether Protocol v2.0 Centralized Externality Management

Generated Specification

April 30, 2025
Version 2.0 (Draft)

Contents

1 Introduction

1.1 Scope

This document specifies the Aether Protocol version 2.0, a **centralized** system designed for the management and regulation of measurable, cross-border economic externalities arising from environmental impacts. This protocol operates under the governance of a single, trusted Central Authority (CA) and relies on standardized reporting, centralized verification, and contractually enforced compliance mechanisms among participating Agents (e.g., nations, corporations, regional bodies). It supersedes previous decentralized concepts under the Aether name.

1.2 Objectives

- Establish a trusted, transparent, and efficient registry for cross-border externality commitments and impacts.
- Provide standardized methods (EFS) for quantifying the economic impacts of specific environmental actions.
- Implement a fair and predictable mechanism for adjusting obligations (e.g., quotas, fees, compensation liabilities) based on verified impact deltas.
- Facilitate auditable tracking of commitments, impacts, adjustments, and compliance.
- Support legally enforceable dispute resolution and compliance frameworks.
- Reduce negotiation overhead and increase regulatory certainty for participants.

1.3 Critical Sectors (Potential Applications)

This protocol is potentially applicable to sectors with significant, measurable cross-border environmental externalities, including but not limited to:

- Industrial Emissions (e.g., SO_x, NO_x, specific greenhouse gases linked to localized economic impacts).
- Water Resource Management (e.g., pollution affecting downstream users, water extraction impacting neighbours).
- Agriculture (e.g., nutrient runoff).
- Waste Management (e.g., transboundary waste shipment impacts).

2 System Architecture

The Aether Protocol v2.0 employs a centralized architecture coordinated by the Central Authority (CA). Participating Agents interact with the CA primarily through defined Application Programming Interfaces (APIs) for reporting, querying status, and managing disputes. The CA maintains the canonical state, performs verification and calculations, and manages the audit trail, potentially leveraging a permissioned ledger for enhanced transparency and immutability of logs (see Figure 1).

The CA acts as the central hub, receiving commitments and data via APIs, performing verification, calculating adjustments based on EFS rules, managing disputes, maintaining the audit log, and coordinating enforcement based on contractual agreements.

Figure: Conceptual Diagram Placeholder

A diagram would show a central **Central Authority (CA)** node. Several **Agent** nodes connect to the CA, indicating submission of Commitments/Reports and reception of Obligations/Notifications. A cloud symbol labeled **APIs** connects to the CA, indicating the interface method. An **Audit Ledger** (database/cylinder symbol) connects below the CA, showing where records are kept. An **External Data / Auditors** node connects to the CA, indicating input for verification and potential external auditing access. Arrows indicate the primary directions of interaction (Agents report TO CA, CA notifies Agents, CA records TO Ledger, etc.).

Figure 1: Conceptual Aether Protocol v2.0 Centralized Architecture.

3 Core Concepts & Definitions

- **Agent:** A participating entity (e.g., nation, corporation, regional authority) registered with the Central Authority and interacting via defined APIs, identified by a unique `AgentID`.
- **Central Authority (CA):** The single, trusted entity governing the protocol, responsible for data validation, calculation, logging, and coordination.
- **Externality Function Specification (EFS):** A precise, CA-approved definition module for a *specific* cross-border externality. Includes:
  - `EFS_ID`: Unique identifier.
  - `Description`: Clear language description.
  - `ActionType`: Measurable action by the causer (e.g., emission rate). May involve Quotas.
  - `ImpactType`: Measurable economic impact on the affected party (e.g., cost increase).
  - `AttributionModel`: CA-managed algorithm linking ActionType to ImpactType.
  - `VerificationDataSpec`: Required data inputs and sources managed by the CA.
  - `AdjustmentRule`: CA-managed formula for obligation adjustments (See Appendix A).
  - `API Schema`: Definition for reporting related data (See Section 5).
- **Verifiable Economic Impact:** The quantified economic consequence (ΔCost or ΔUtility) calculated by the CA using an approved EFS and verified data.
- **Commitment:** A formal pledge submitted by an Agent to the CA via API regarding future actions.
- **Epoch:** A discrete time period (e.g., monthly, quarterly) for reporting and adjustment calculations, defined by the CA.
- **Audit Log:** The official, tamper-evident record maintained by the CA, potentially on a permissioned ledger.
- **Obligation:** A requirement placed on an Agent by the CA (e.g., adhere to a quota, pay compensation) resulting from the Adjustment Layer calculation.
- **Contractual Agreement:** The legally binding agreement between each Agent and the Central Authority defining terms of participation, enforcement, etc.

4 Components

4.1 Authority Layer

- **Entity:** A single, trusted governance entity established by treaty or mutual agreement. Its legal standing and operational charter must be clearly defined.
- **Roles & Responsibilities:**
  - *Governance & EFS Management:* Defines, approves, and updates EFSs; manages parameter governance (Section 6).
  - *Agent Registration & Management:* Manages Agent registry and API credentials.
  - *Commitment Registry:* Receives and logs Agent commitments via secure APIs.
  - *Data Verification:* Validates submitted data and/or procures external data per EFS requirements.
  - *Calculation Engine:* Executes EFS `AdjustmentRules`.
  - *Obligation Issuance:* Communicates calculated adjustments to Agents.
  - *Dispute Resolution Management:* Administers the dispute process (Section 4.5).
  - *Enforcement Coordination:* Monitors compliance and initiates contractually agreed enforcement actions (Section 4.6).
  - *Reputation Management:* Calculates and publishes Agent reputational scores.
  - *Audit Log Maintenance:* Maintains the secure, timestamped Audit Log (Appendix D).

4.2 Commitment Layer

- **Function:** Enables Agents to make formal, cryptographically signed pledges to the CA regarding future actions.
- **Mechanism:** Agents submit commitment data via standardized, authenticated APIs (Appendix C).
- **Data Integrity:** Commitments digitally signed by the Agent. CA verifies signature and logs the commitment. Use of content hashes recommended.
- **Logging:** CA logs verified commitments in the Audit Log.

4.3 Verification Layer

- **Function:** The CA verifies reported Agent actions and resulting environmental/economic impacts against EFS criteria and commitments. Decentralized oracles are explicitly removed.
- **Process:**
  1. CA collects necessary data (self-reported via API, independent monitoring, designated auditors/sources).
  2. CA validates data authenticity and integrity.
  3. CA compares verified actions against prior commitments; flags discrepancies.
  4. CA applies EFS `AttributionModel` to calculate Verifiable Economic Impacts ($V_{i \to j,t}$).
  5. All steps, data references (hashes), impacts, and outcomes recorded in the Audit Log by the CA.
- **Transparency:** Achieved through access-controlled viewing of the detailed Audit Log.

4.4 Adjustment Layer

- **Function:** The CA centrally calculates and issues adjustments based on the delta-only principle.
- **Process:** (Runs periodically, e.g., end of Epoch $t + k$)
  1. CA Calculation Engine retrieves verified impact data ($V_{i \to j,t}$) for Epoch t.
  2. Aggregates impacts, calculates total delta (Δ_t).
  3. Applies relevant EFS `AdjustmentRule` (e.g., Appendix A).
  4. Records resulting `Obligations` in the Audit Log.
  5. Notifies relevant Agents of new obligations via secure channels/APIs.

4.5 Dispute Resolution Layer

- **Function:** Formal mechanism for Agents to challenge CA decisions or data accuracy.
- **Mechanism:**
  1. Agent submits dispute claim to CA via defined procedure/API with evidence.
  2. CA logs dispute in an "Arbitration Registry" (within Audit Log or separate module).
  3. CA follows internal review or refers to external arbitration as per Agent contracts.
  4. Final resolution determined by designated authority (CA or arbitrator).
  5. Resolution outcome recorded in the Arbitration Registry / Audit Log.
- **Enforceability:** Resolutions intended to be binding and legally enforceable via the contractual agreements.

4.6 Enforcement Layer

- **Function:** Ensures compliance through non-crypto-economic means (replaces staking/slashing).
- **Mechanisms:**
  - **Contractual Agreements:** Legally binding agreements between Agents and CA define obligations and consequences.
  - **Compliance Monitoring:** CA tracks adherence to commitments, reporting, quotas, payments.
  - **Reputational Scoring:** CA maintains and publishes Agent reputation scores based on compliance history.
  - **Formal Non-Compliance Reporting:** CA issues notices, applies contractual penalties, reports breaches as per agreements.
  - **Linkage to Existing Frameworks:** May leverage existing international legal/trade frameworks if stipulated in contracts.

5 Externality Function Specifications (EFS)

EFS definitions are managed and curated by the Central Authority.

- **Definition:** As defined conceptually in Section 3, formally approved and maintained by the CA.
- **Data Types:** Must clearly specify expected data types and units for inputs/outputs. Standardized types (ISO units, financial standards) preferred. See Appendix B.
- **API Schemas:** CA publishes clear API schemas (e.g., OpenAPI/Swagger) defining structure, types, units, validation rules for data submission related to each EFS. See Appendix C.

6 Parameter Governance

Governance of protocol parameters (e.g., EFS coefficients, baselines, thresholds) is managed by the Central Authority.

- **Process:** Formal proposal and review process managed by CA.
- **RFC Mechanism:** Consultative RFC process used for input on significant parameter changes or new EFS adoption.
- **Transparency:** Decisions and rationale documented and communicated, potentially via Audit Log entries.
- **Authority Structure:** Specific decision mechanism depends on CA's defined governance (e.g., board vote, expert panel).

7 Security Considerations

Security focus shifts to the Central Authority's infrastructure and processes.

- **CA Infrastructure Security:** Protecting CA servers, databases, and internal networks.
- **API Security:** Robust authentication (e.g., OAuth2, API Keys), authorization, input validation, rate limiting, TLS encryption.
- **Data Integrity:** Use of digital signatures, cryptographic hashing, secure Audit Log maintenance (potentially DLT).
- **Auditability:** Ensuring Audit Log completeness, accuracy, and appropriate access control.
- **Insider Threat Mitigation:** Access controls, separation of duties, monitoring within the CA.
- **Legal & Contractual Security:** Enforceability of underlying agreements.

8 Operational Considerations

- **CA Establishment:** Requires significant legal, political, and technical effort.
- **Agent Registration**: Formal process including contractual agreements and API credentialing.
- **System Maintenance:** Ongoing maintenance of CA infrastructure, APIs, databases/ledger. Regular security audits.
- **Data Source Management:** Establishing and maintaining reliable data/auditor relationships.

9 Open Questions & RFC Areas (Centralized Context)

While centralized, refinement still requires input:

- **RFC-C01:** Specific EFS Definitions & Attribution Models: Formalizing initial EFSs.
- **RFC-C02:** CA Governance Structure: Detailed definition of CA decision-making processes.
- **RFC-C03:** Contractual Agreement Template: Standard terms for Agent participation and enforcement.
- **RFC-C04:** API Specification Details: Finalizing OpenAPI schemas, authentication methods.
- **RFC-C05:** Audit Log Technology Access Policy: Choice of ledger/database, detailed permission model.
- **RFC-C06:** Dispute Resolution Procedures: Detailed rules for internal review and external arbitration integration.
- **RFC-C07:** Reputation Scoring Algorithm: Specific formula and consequences.
- **RFC-C08:** Data Privacy Rules: Handling sensitive data reported by Agents or collected by CA.
- **RFC-C09:** Parameter Tuning Cadence: Process and frequency for reviewing EFS parameters.
- **RFC-C10:** Integration with Legal Systems: Mechanisms for enforcing contractual obligations across jurisdictions.

10 Conclusion

Aether Protocol v2.0 proposes a centralized framework for managing cross-border economic externalities, leveraging a trusted Central Authority, standardized EFS definitions, secure APIs, and contractual enforcement. By centralizing verification and adjustment, it aims for efficiency and clear accountability, while relying on robust governance, transparent audit logs, and strong legal agreements for trust and compliance. Its successful implementation depends on establishing a credible and well-governed Central Authority and securing binding commitments from participating Agents.

A Example Adjustment Rules

These examples illustrate `AdjustmentRule` logic executed by the Central Authority's Calculation Engine.

A.1 Contribution-Weighted Delta Adjustment (Quota Regulation)

A.1.1 Purpose

CA adjusts Agent quotas based on the change in total verified economic impact, weighted by each Agent's verified contribution.

A.1.2 Assumptions for EFS

- EFS involves controllable `ActionType` (e.g., SO_2 emissions) with quotas Q_i.
- CA goal is stabilizing/reducing total impact V_{total}.
- Verification lag k exists. Adjustments decided end of Epoch $t+k$ apply to Epoch $t+k+1$.

A.1.3 Parameters defined in Rule (Managed by CA)

- `BaselineImpact` (V_{baseline})
- `SensitivityFactor` (α)
- `MinimumQuota` (Q_{min})
- `MaximumQuota` (Q_{max})

A.1.4 Algorithm Steps (Executed by CA)

1. **Retrieve Verified Data (Epoch t):** $V_{\text{total},t}$, $V_{i\to\text{total},t}$, $V_{\text{total},t-1}$ (or V_{baseline}), current quota $Q_{i,t+k}$.
2. **Calculate Delta:** $\Delta_t = V_{\text{total},t} - V_{\text{total},t-1}$.
3. **Determine Preliminary New Quota $Q'_{i,t+k+1}$:**
   - If $\Delta_t \leq 0$: $Q'_{i,t+k+1} = Q_{i,t+k}$ (V1: Unchanged).
   - If $\Delta_t > 0$: (Quota tightening required)
     (a) Share $s_{i,t} = V_{i\to\text{total},t}/V_{\text{total},t}$ (handle $V_{\text{total},t} = 0$).
     (b) Relative increase $r_t = \Delta_t/V_{\text{total},t-1}$ (handle $V_{\text{total},t-1} = 0$).
     (c) Reduction factor $f_{i,t} = \alpha \times r_t \times s_{i,t}$.
     (d) $Q'_{i,t+k+1} = Q_{i,t+k} \times \max(0, 1 - f_{i,t})$.
4. **Apply Floor/Ceiling:** $Q_{i,t+k+1} = \max(Q_{\text{min}}, \min(Q_{\text{max}}, Q'_{i,t+k+1}))$.
5. **Record & Notify:** CA logs $Q_{i,t+k+1}$ as the Obligation and notifies Agent i.

A.2 Direct Compensation Delta Rule

A.2.1 Purpose

CA requires Agents causing an increase in aggregate impact to compensate affected Agents.

A.2.2 Assumptions for EFS

- EFS links causer i to affected j with impact $V_{i \to j,t}$.
- CA goal is compensating for *increases* in harm (V_{total}).
- Verification lag k exists.

A.2.3 Parameters defined in Rule (Managed by CA)

- `BaselineImpact` (V_{baseline})
- `CompensationMultiplier` (γ)

A.2.4 Algorithm Steps (Executed by CA)

1. **Retrieve Verified Data (Epoch t):** $V_{\text{total},t}$, $V_{i \to \text{total},t}$, $V_{i \to j,t}$, $V_{\text{total},t-1}$ (or V_{baseline}).
2. **Calculate Delta:** $\Delta_t = V_{\text{total},t} - V_{\text{total},t-1}$.
3. **Determine Obligation:**
   - If $\Delta_t \leq 0$: No compensation obligation. End.
   - If $\Delta_t > 0$: Proceed for each causer i.
4. **Calculate and Allocate Compensation** (for each causer agent i where $V_{i \to \text{total},t} > 0$):
   (a) Share $s_{i,t} = V_{i \to \text{total},t} / V_{\text{total},t}$.
   (b) Allocated delta $\Delta_{i,t}^{\text{allocated}} = \Delta_t \times s_{i,t}$.
   (c) Total required from i: $C_{i,t} = \gamma \times \Delta_{i,t}^{\text{allocated}}$.
   (d) Distribute to affected j (where $V_{i \to j,t} > 0$, handle division by zero if $V_{i \to \text{total},t} = 0$):
   $$C_{i \to j,t} = C_{i,t} \times \frac{V_{i \to j,t}}{V_{i \to \text{total},t}}$$
   (e) Record Obligation: CA logs "Agent i owes $C_{i \to j,t}$ to Agent j for Epoch t, due by end of Epoch $t + k + 1$" and notifies parties.

B Data Format Definitions (Conceptual Examples)

Data exchanged via APIs should adhere to well-defined formats, ideally JSON.

```json
{
  "agentId": "AGENT_PUBKEY_OR_ID",
  "efsId": "EFS_SO2_IMPACT_ZONE_B",
  "epochId": 202503, // e.g., YYYYMM
  "commitment": {
    "actionType": "SO2_EMISSION_RATE",
    "value": 95.5,
    "unit": "tonnes/day"
  },
  "commitmentHash": "sha256_hash_of_commitment_object", // Optional
  "timestamp": "2025-06-15T10:00:00Z", // ISO 8601
  "signature": "agent_digital_signature_of_payload"
}
```

Listing 1: Example: Commitment Payload Format

```
{
  "agentId": "AFFECTED_AGENT_ID",
  "efsId": "EFS_SO2_IMPACT_ZONE_B",
  "epochId": 202503,
  "impactReport": {
    "impactType": "HEALTHCARE_COST_INCREASE",
    "value": 15000.00,
    "currency": "USD",
    "attribution": [ // Optional: Agent's claimed attribution
      { "causerAgentId": "CAUSER_AGENT_ID", "percentage": 75.0 }
    ],
    "evidenceHash": "hash_of_supporting_documents" // Link/Hash
  },
  "timestamp": "2025-07-01T14:30:00Z",
  "signature": "agent_digital_signature_of_payload"
}
```

Listing 2: Example: Impact Report Payload Format (Self-Reported)

C API Endpoints (Conceptual Examples)

The CA exposes secure, authenticated RESTful APIs.

```
# --- Commitments ---
# Submit a commitment for an epoch
POST /v1/commitments
Authorization: Bearer <AGENT_TOKEN>
Content-Type: application/json
Body: (See lst:commit_format_v2)
# Response: 201 Created { "commitmentId": "...", "status": "LOGGED" }

# Retrieve commitments
GET /v1/commitments?agentId=<AGENT_ID>&epochId=<EPOCH_ID>
Authorization: Bearer <AGENT_TOKEN_OR_CA_TOKEN>
# Response: 200 OK [ { ...commitment details... } ]

# --- Data Reporting ---
# Submit self-reported data (e.g., impacts)
POST /v1/reports/impact
Authorization: Bearer <AGENT_TOKEN>
Content-Type: application/json
Body: (See lst:impact_format_v2)
# Response: 202 Accepted { "reportId": "...", "status": "PENDING_VERIFICATION" }

# --- Obligations & Status ---
# Get obligations for an agent for a past impact epoch
GET /v1/obligations/{agentId}?impactEpochId=<EPOCH_ID>
Authorization: Bearer <AGENT_TOKEN_OR_CA_TOKEN>
# Response: 200 OK { "impactEpochId": ..., "obligations": [ { "type": "QUOTA", "value": ..., "unit": ..., "appliesToEpoch": ... }, { "type": "COMPENSATION", "amount": ..., "currency": ..., "beneficiary": ..., "dueEpoch": ... } ] }

# Get current status and reputation score
GET /v1/status/{agentId}
Authorization: Bearer <AGENT_TOKEN_OR_CA_TOKEN>
# Response: 200 OK { "reputationScore": ..., "complianceStatus": "..." }

# --- Disputes ---
# Submit a dispute
POST /v1/disputes
```

```
Authorization: Bearer <AGENT_TOKEN>
Content-Type: application/json
Body: { "epochId": ..., "subjectType": "VERIFICATION | ADJUSTMENT", "subjectId": "...", "reason": "...", "evidenceReference": "..." }
# Response: 202 Accepted { "disputeId": "...", "status": "RECEIVED" }

# Get dispute status
GET /v1/disputes/{disputeId}
Authorization: Bearer <AGENT_TOKEN_OR_CA_TOKEN>
# Response: 200 OK { ...dispute details and status... }
```

Listing 3: Conceptual API Endpoint Definitions

D Sample Audit Log Entries

CA maintains detailed logs, potentially using a structured format like JSON on a permissioned ledger.

```
{
  "log_entry_id": "uuid_or_ledger_tx_id_1",
  "timestamp": "2025-07-15T11:05:30Z",
  "event_type": "COMMITMENT_LOGGED",
  "details": {
    "agent_id": "AGENT_ID_1",
    "efs_id": "EFS_SO2_IMPACT_ZONE_B",
    "epoch_id": 202504,
    "commitmentHash": "sha256_hash...",
    "original_signature": "agent_signature..."
  },
  "status": "SUCCESS",
  "authority_sig": "ca_signature_of_log_entry"
}

{
  "log_entry_id": "uuid_or_ledger_tx_id_2",
  "timestamp": "2025-08-01T09:15:00Z",
  "event_type": "VERIFICATION_COMPLETED",
  "details": {
    "efs_id": "EFS_SO2_IMPACT_ZONE_B",
    "epoch_id": 202503,
     "verified_impacts": [
      { "causer": "AGENT_ID_A", "affected": "AGENT_ID_B", "amount": 16500.00, "currency": "USD" }
    ],
    "data_sources_hash": "hash_of_input_data_refs"
  },
  "status": "SUCCESS",
  "authority_sig": "ca_signature_of_log_entry"
}

{
  "log_entry_id": "uuid_or_ledger_tx_id_3",
  "timestamp": "2025-08-01T10:00:00Z",
  "event_type": "ADJUSTMENT_CALCULATED",
   "details": {
    "efs_id": "EFS_SO2_IMPACT_ZONE_B",
    "epoch_id": 202503, // Impact epoch
    "delta_impact": 1500.00, // Currency implicit from EFS/context
    "obligations_created": [
      { "obliged": "AGENT_ID_A", "type": "QUOTA_ADJUST", "value": "-5.0%", "applies_to_epoch": 202505 },
```

```
      { "obliged": "AGENT_ID_A", "type": "COMPENSATE", "amount": 1500.00, "currency": "USD", "beneficiary": "AGENT_ID_B", "due_epoch": 202505 }
    ]
  },
  "status": "SUCCESS",
  "authority_sig": "ca_signature_of_log_entry"
}
```

Listing 4: Conceptual Audit Log Entry Format (JSON-like)